Technical Report: Personalized Movie Recommendation System

1. Problem Understanding & Motivation

Recommendation engines are crucial to modern digital entertainment platforms. The objective of this project is to build a personalized movie recommendation system using historical rating logs from the Netflix Prize Dataset, enabling the platform to improve content discovery, increase user engagement, and retain subscribers.

The core challenge involves:

- **High Data Sparsity**: Real-world user-item rating matrices are extremely sparse, requiring robust dimensional reduction models.
- **Accuracy vs. Ranking**: Traditional metrics like RMSE evaluate rating prediction accuracy, but recommendation lists are evaluated on ranking quality (retrieving relevant content). We evaluate both using RMSE and MAP@10.
- **Explainability & Transparency**: Users trust recommendations when they understand why they were generated. We address this using item-based similarity explanations.
- **Cold Start**: Recommending to new users with zero history or rating new movies with zero interactions.

2. Exploratory Data Analysis (EDA)

We processed a dense subset of the Netflix Prize Dataset to train SVD and KNN models. The structural highlights of the subset include:

- **Total Ratings**: 5,702,695 reviews
- **Unique Users**: 21,045 active users (minimum 200 ratings)
- **Unique Movies**: 1,287 popular movies (minimum 2,000 ratings)
- **Matrix Density**: 21.04% (Matrix Sparsity: 78.96%)

Key Insights:

1. **Rating Bias**: Rating distribution is heavily skewed towards high ratings (4 and 5 stars represent over 60% of all ratings, with a mean rating of 3.73). This indicates a strong positive selection bias.

2. **User Activity**: Power-users rate hundreds of movies, causing a high-density core in the rating matrix. This helps local neighborhood models achieve better precision.

3. **Movie Popularity**: Blockbuster titles (e.g. *Lord of the Rings*, *Gladiator*) dominate the total rating counts, demonstrating a typical power-law long-tail distribution.

---

3. Methodology & Model Architectures

We implemented and compared two distinct recommendation paradigms:

A. Matrix Factorization (Singular Value Decomposition - SVD)

The SVD model factorizes the rating matrix `R` into user latent factors `P` and item latent factors `Q`, predicting ratings as:

$\hat{r}_{u,i} = \mu + b_u + b_i + p_u^T q_i$

Where:

- μ is the global mean rating.
- `b_u, b_i` are user and item biases.
- `p_u, q_i` are `f`-dimensional latent feature vectors.

The model is optimized using Stochastic Gradient Descent (SGD) with L2 regularization to minimize squared error over historical ratings:

$\min(b, p, q) \sum_{(u,i)\in K} (r_{u,i} - \hat{r}_{u,i})^2 + \lambda (b_u^2 + b_i^2 + ||p_u||^2 + ||q_i||^2)$

B. Item-Based Collaborative Filtering (KNN with Means)

A neighborhood model that calculates similarity weights between items based on ratings overlapping profiles. The predicted rating is calculated as:

$\hat{r}_{u,i} = \mu_i + [(\sum \text{ over } j \text{ in } N_u(i)) \text{ sim}(i, j) \cdot (r_{u,j} - \mu_j)] / [(\sum \text{ over } j \text{ in } N_u(i)) |\text{sim}(i, j)|]$

Where `sim(i, j)` is the cosine similarity between items `i` and `j`, `N_u(i)` is the set of items rated by user `u` that are most similar to item `i`, and μ_i, μ_j denote item mean ratings. This matches the item-based `KNNWithMeans` setup used in the notebooks.

---

4. Evaluation & Results

We split the dataset into an 80-20 train-test split and evaluated on two mandatory metrics:

1. **RMSE (Root Mean Squared Error)**:

   $$RMSE = \sqrt{[(1 / |T|) \sum_{(u,i)\in T} (r_{u,i} - \hat{r}_{u,i})^2]}$$

2. **MAP@10 (Mean Average Precision @ 10)**: Evaluates recommendation ranking quality. A movie is relevant if its actual rating in the test set is ≥ 3.5.

Additional Metrics (Optional)

To provide a highly rigorous evaluation, we additionally implemented and reported:

- **MAE (Mean Absolute Error)**: Measures average absolute deviation between predicted and actual ratings, treating all errors equally.

  $$MAE = (1 / |T|) \sum_{(u,i)\in T} |r_{u,i} - \hat{r}_{u,i}|$$

- **Precision@10**: Percentage of top 10 recommended items that are relevant (rating ≥ 3.5).
- **Recall@10**: Percentage of the user's total relevant test items retrieved in the top 10 list.
- **NDCG@10 (Normalized Discounted Cumulative Gain)**: Evaluates the graded relevance of items taking position into account (relevant items at the top are weighted higher).
- **Hit Rate@10**: Percentage of users who get at least one relevant recommendation in their top 10.
- **Coverage**: Percentage of users for whom the model is able to generate top 10 recommendations.
- **Diversity (Intra-List Diversity)**: Average difference (`1 - similarity`) between recommended items, ensuring users receive a varied feed.

Experimental Comparisons & Results:

Metric	Matrix Factorization (SVD)	Item-Based (KNN)
Test RMSE (Lower is Better)	**0.8160**	0.8553
Test MAE (Lower is Better)	**0.6368**	0.6687
MAP@10 (Higher is Better)	**0.7427**	0.7079
Precision@10 (Higher is Better)	**0.8051**	0.7814
Recall@10 (Higher is Better)	**0.3388**	0.3272
NDCG@10 (Higher is Better)	**0.8337**	0.8081
Hit Rate@10 (Higher is Better)	**99.94%**	99.92%

User Coverage (Higher is Better)	**99.96%**	99.96%
Intra-List Diversity (Higher is Better)	**0.0477**	0.0452
Training Time (s)	**8.92s**	225.86s
Inference Time (s)	**0.31s**	4.85s

Evaluation Justification & Performance Trade-offs

1. **Rating Accuracy vs. Ranking Performance**:
   - Rating prediction metrics (RMSE/MAE) measure how closely predictions match ratings. SVD minimizes squared error, so it excels here.
   - Ranking metrics (Precision@K, Recall@K, NDCG) measure how well items are *sorted*. SVD achieves **83.37% NDCG@10** and **74.27% MAP@10**, outperforming KNN. This shows that rating prediction accuracy correlates strongly with ranking performance for latent-factor systems on dense matrices.
2. **Sparsity & Coverage**: Both models achieved **99.96% User Coverage** on the evaluation set (only omitting users with no positive ratings in the test set). SVD has complete coverage over the item/user space, while KNN is limited when items have no ratings (a key bottleneck for neighborhood approaches).
3. **Diversity Trade-off**: The SVD recommendations have slightly higher diversity (0.0477) than KNN (0.0452), showing SVD covers a slightly wider range of item profiles. Generally, models optimizing for accuracy tend to favor popular, safe items (lower diversity), whereas specific diversity-oriented loss functions can yield more novel recommendations at the cost of slight accuracy degradation.

---

5. Explainable AI & Cold Start Strategies

A. Explainability Framework

While SVD produces high-quality predictions, it lacks transparency because latent dimensions (e.g., factor 3) are not human-interpretable. To resolve this, we leverage the item-similarity weights computed by the KNN model. For an SVD recommended movie M_{rec}, we retrieve its most similar movies from the cosine similarity matrix, find the highest-rated movie $M_{watched}$ in the user's history, and output:

> *"Because you gave '{M_watched}' a rating of {rating} (Similarity: {score}%)"*

B. Cold Start Strategies

- **New Users**: When a user has no historical ratings, collaborative filtering fails. We resolve this in the API by falling back to a **Popularity-based Recommender** returning high-average rated movies that have received at least 2,000 ratings.
- **New Movies**: Newly added content with zero ratings is given initial similarity weights by checking genre overlap with existing popular movies. We also propose an ε-greedy exploration strategy to dynamically promote new titles to a random subset of users to acquire initial feedback.

---

6. Deployment Architecture

We deployed the solution as a decoupled full-stack application:

- **API Backend**: FastAPI (Python) serving uvicorn endpoints for search, statistics, similarities, and personalized recommendations with similarity-based explanations.
- **Interactive UI Dashboard**: React SPA styled with custom dark-mode glassmorphism styling, loading sample users and serving charts plotted directly using custom SVG code.